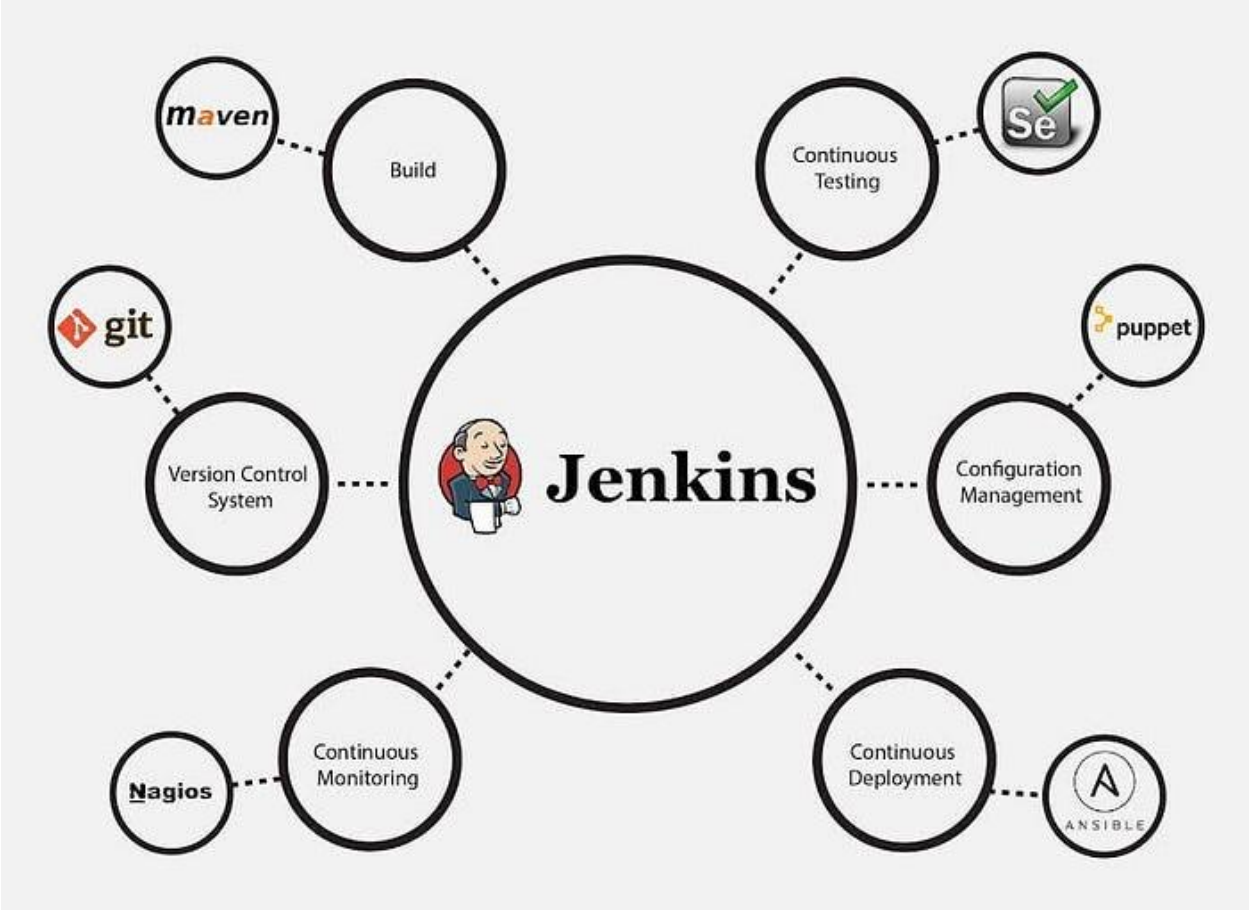
Jenkins

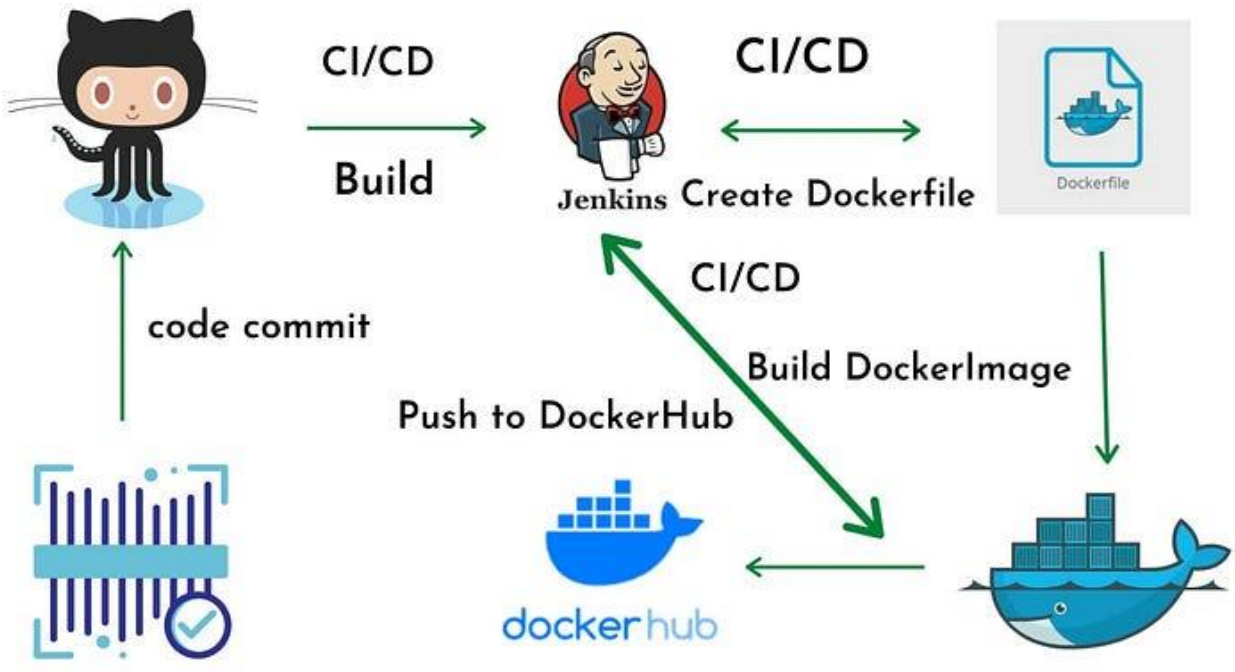
What Is Jenkins?

Jenkins is an open-source solution comprising an automation server to enable continuous integration and continuous delivery (CI/CD), automating the various stages of software development such as build, test, and deployment.

Its a Java-based open-source automation platform with plugins designed for continuous integration. It is used to continually create and test software projects, making it easier for developers and devops engineers to integrate changes to the project and for consumers to get a new build. It also enables you to release your software continuously by interacting with various testing and deployment methods.



What is Continuous Integration?



Continuous Integration is preferable to a Nightly Build and Integration process, run at day's end when everyone has gone home (freeing server resources). Nightly integration is limited, occurring only once per day, as opposed to the continuous process of CI. Developers agree that nightly integration is useful in situations where the build process takes such an inordinately large amount of time that it is best conducted when fewer people are accessing the servers.

What Is Jenkins Used For?

Jenkins software's popularity stems from its ability to track and monitor repetitive activities that emerge throughout a project's development. For example, if your team is working on a project, Jenkins will continually test your builds and alert you to any mistakes early in the process. Its top use cases include:

1. Deploying code into production

If all of the tests developed for a feature or release branch are green, Jenkins or another CI system may automatically publish code to staging or production. This is often referred to as continuous deployment. Changes are done before a merging action can also be seen. One may do this in a dynamic staging environment. Then it's distributed to a central staging system, a pre-production system, or even a production environment when combined.

2. Enabling task automation:

Another instance in which one may use Jenkins is to automate flows and tasks. If a developer is working on several environments, they will need to install or upgrade an item on each of them. If the installation or update requires more than 100 steps to complete, it will be error-prone to do it manually. Instead, you can write down all the steps needed to complete the activity in Jenkins. It will take less time, and you can complete the installation or update without difficulty.

3. Reducing the time it takes to review a code:

Jenkins is a CI system that may communicate with other DevOps tools and notify users when a merge request is ready to merge. This is typically the case when all tests have been passed and all other conditions have been satisfied. Furthermore, the merging request may indicate the difference in code coverage. Jenkins cuts the time it takes to examine a merge request in half. The number of lines of code in a

component and how many of them are executed determines code coverage. Jenkins supports a transparent development process among team members by reducing the time it takes to review a code.

4. Driving continuous integration:

Before a change to the software can be released, it must go through a series of complex processes. The Jenkins pipeline enables the interconnection of many events and tasks in a sequence to drive continuous integration. It has a collection of plugins that make integrating and implementing continuous integration and delivery pipelines a breeze. A Jenkins pipeline's main feature is that each assignment or job relies on another task or job.

On the other hand, continuous delivery pipelines have different states: test, build, release, deploy, and so on. These states are inextricably linked to one another. A CD pipeline is a series of events that allow certain states to function.

5. Increasing code coverage:

Jenkins and other CI servers may verify code to increase test coverage. Code coverage improves as a result of tests. This encourages team members to be open and accountable. The results of the tests are presented on the build pipeline, ensuring that team members adhere to the guidelines. Like code review, comprehensive code coverage guarantees that testing is a transparent process for all team members.

6. Enhancing coding efficiency:

Jenkins dramatically improves the efficiency of the development process. For example, a command prompt code may be converted into a GUI button click using Jenkins. One may accomplish this by encapsulating the script in a Jenkins task. One may parameterize Jenkins tasks to allow for customization or user input. Hundreds of lines of code can be saved as a result.

Further, it supports manual testing where necessary without switching environments. When code is hosted locally, it does not always work well when pushed to a central system on a private or public cloud. This occurs because things change by the time they push. Continuous integration on Jenkins allows for manual testing that compares code to the current state of a code base in a production-like environment.

7. Simplifying audits:

When Jenkins tasks run, they collect console output from stdout and stderr parameters. This makes troubleshooting using Jenkins extremely straightforward. You may assess run timing and find the slowest step utilizing the time stamper plugin, allowing you to tweak the performance of each operation.

8. Using Slack for synchronization:

A major Jenkins use case is its interoperability with Slack. A centralized communication platform is a must-have for large teams,

and one of the most popular platforms for this purpose is Slack. Jenkins may be integrated with Slack, allowing communication such as triggered activities, their times, users' names, and outcomes to be shared with others.

➢Jenkins Core Concepts

Jenkins Controller (Formerly Master)

The Jenkins architecture supports distributed builds. One Jenkins node functions as the organizer, called a Jenkins Controller. This node manages other nodes running the Jenkins Agent. It can also execute builds, although it isn't as scalable as Jenkins agents.

The controller holds the central Jenkins configuration. It manages agents and their connections, loads plugins, and coordinates project flow.

Jenkins Agent (Formerly Slave)

The Jenkins Agent connects to the Jenkins Controller to run build jobs. To run it, you'll need to install Java on a physical machine, virtual machine, cloud compute instance, Docker image, or Kubernetes cluster.

You can use multiple Jenkins Agents to balance build load, improve performance, and create a secure environment independent of the Controller.

Jenkins Node

A Jenkins node is an umbrella term for Agents and Controllers, regardless of their actual roles. A node is a machine on which you can build projects and pipelines. Jenkins automatically monitors the health of all connected nodes, and if metrics go below a threshold, it takes the node offline.

Jenkins Project (Formerly Job)

A Jenkins project or task is an automated process created by a Jenkins user. The plain Jenkins distribution offers a variety of build tasks that can support continuous integration workflows, and more are available through a large ecosystem of plugins.

Jenkins Plugins

Plugins are community-developed modules you can install on a Jenkins server. This adds features that Jenkins doesn't have by default. You can install/upgrade all available plugins from the Jenkins dashboard.

Jenkins Pipeline

A Jenkins Pipeline is a user-created pipeline model. The pipeline includes a variety of plugins that help you define step-by-step actions in your software pipeline. This includes:

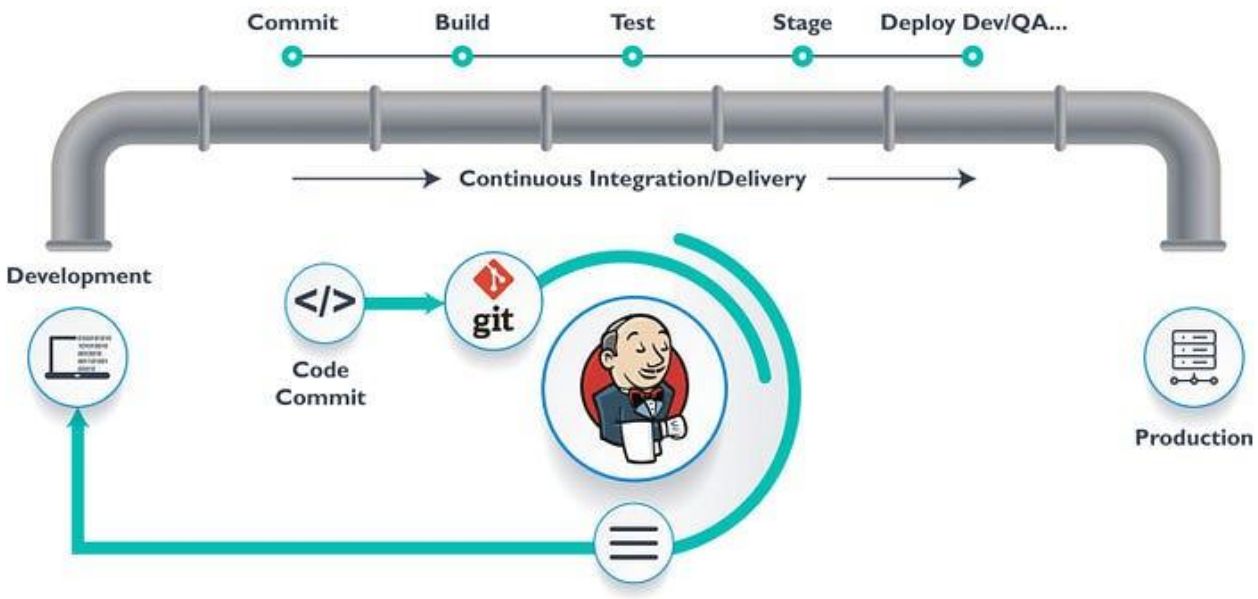
- Automated builds.

- Multi-step testing.
- Deployment procedures.
- Security scanning

You can create pipelines directly in the user interface, or create a “Jenkinsfile” which represents a pipeline as code. Jenkinsfiles use a Groovy-compatible text-based format to define pipeline processes, and can be either declarative or scripted.

>>>Lets understand them in depth :

What Is a Jenkins Pipeline?



Pipelines are needed to run Jenkins. A pipeline is a set of steps the Jenkins server will execute to complete the CI/CD process’s necessary

tasks. In the context of Jenkins, a pipeline refers to a collection of jobs (or events) connected in a specific order. It is a collection of plugins that allow the creation and integration of Continuous Delivery pipelines in Jenkins.

The 'Pipeline Domain-Specific Language (DSL)' syntax also provides a collection of tools for modeling basic and sophisticated delivery pipelines 'as code.' Every task in the Jenkins pipeline is reliant on one or more events in some way.

Jenkins Pipelines comprise a powerful technology that includes a set of tools for hosting, monitoring, compiling, and testing code or code modifications across various tools such as:

- Continuous integration server (Bamboo, Jenkins, TeamCity, CruiseControl, and others)
- Source control software (e.g., SVN, CVS, Mercurial, GIT, ClearCase, Perforce, and others)
- Build tools (Make, Ant, Ivy, Maven, Gradle, and others)
- Automation testing framework (Appium, Selenium, UFT, TestComplete, and others)

Jenkins pipeline is a user-created continuous delivery pipeline paradigm. It includes several plugins that aid in the different stages, from version control to user-facing delivery. This is important since,

before being released, all software modifications and commits go through a lengthy procedure. The method has three phases: automated building, multi-step testing, and deploying procedures.

There are two methods to construct a pipeline in Jenkins: directly define the pipeline using the user interface or create a Jenkinsfile using the pipeline as code technique. The pipeline process is described in a text file that employs Groovy-compatible syntax. Before constructing a Jenkins pipeline, here are the key terminologies to understand:

1. Multibranch Pipeline

It builds from various branches are automatically grouped to make branch management easier. Jenkins automatically generates a new project when a new branch is pushed to a source code repository. Other plugins can specify different branches, such as a Git branch, a Subversion branch, a GitHub Pull Request, and so on.

2. Archived Jenkins Pipeline

The file archive is secure; you may clear your workspace and perform subsequent builds. For instance, suppose you create the jar/HTML/js file, which is crucial for deployment. Your file gets replaced or erased after another build.

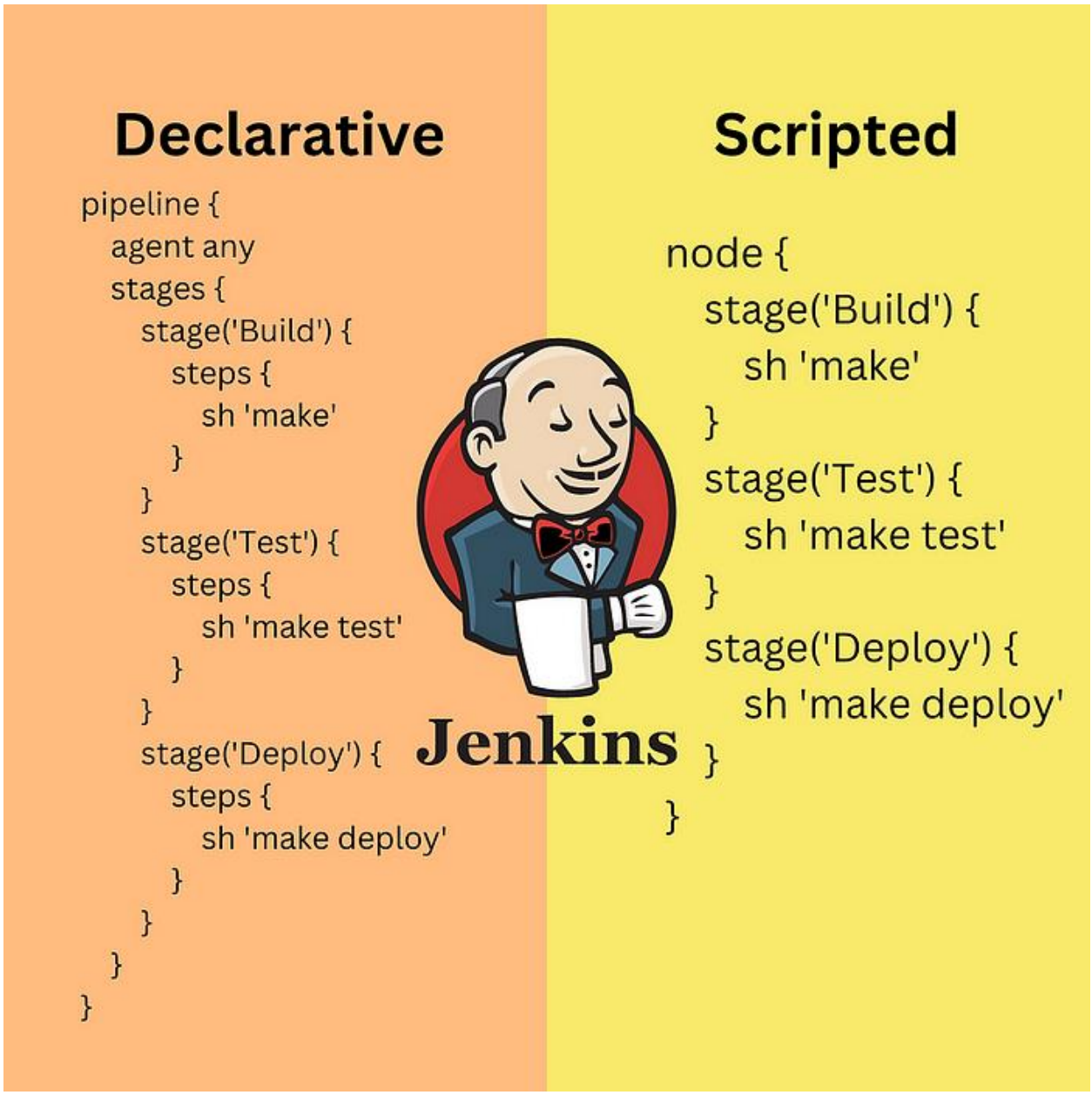
3. Pipeline speed

This is crucial if your pipeline saves huge files or complex data to variables in the script. Jenkins has a “Speed/Durability” label that allows you to maintain variables in scope for future usage while also allowing you to perform steps. However, if your pipelines spend practically all of their time waiting for a few shell/batch scripts to complete, it won’t help. This isn’t a one-size-fits-all capability.

Types of Jenkins Pipeline

There are 2 types of Jenkins Pipeline: -

1. Scripted Pipeline
2. Declarative Pipeline



Scripted Pipeline

Jenkins only allowed the user to write the pipeline as a code, hence a scripted pipeline. Such a pipeline is written in a Jenkins file on the web UI of the Jenkins tool. You can write a scripted pipeline with `DSL (domain specific language)` and use `Groovy based syntax`.

The Job `DSL plugin` provides a solution, and allows you to configure Jenkins jobs as code.

What exactly is `Groovy`*?* It is a programming language that uses the Java virtual machine(JVM).

A `Java virtual machine (JVM)` is a virtual machine that enables a computer to run Java programs as well as programs written in other languages that are also compiled to Java bytecode.

Advantages of Scripted Pipeline

- Scripted pipelines offer a full-fledged programming ecosystem to developers
- Developers can inject Groovy scripts and reference Java APIs inside a scripted pipeline definition
- Blocks of scripted pipelines can be injected inside the script step of a declarative pipeline definition. This enhances cross-pipeline support.

Disadvantage of Scripted Pipeline

- Scripted syntax can be harder to learn for beginners as compared to declarative syntax
- Scripted pipelines don’t offer runtime syntax checking

- Scripted pipelines don't have the When directive, which can be used to skip a stage if a condition isn't met
- It isn't possible to restart a scripted pipeline from a previous stage

Syntax

```
node {
    stage('Build') {
        // Execute Some steps here
    }
    stage('Test') {
        // Execute Some steps here
    }
    stage('Deploy') {
        // Execute Some steps here
    }
}
```

Declarative Pipeline

In many ways, `declarative syntax` represents the modern way of defining pipelines. It is robust, clean, and easy to read and write.

The `declarative` coding approach dictates that the user specifies only what they want to do, not how they want to do it. This makes it easier to create `declarative pipelines`, as compared to `scripted pipelines`, which follow the imperative coding approach.

A `declarative pipeline` offers capabilities for adding logging and error handling, supports the use of conditional statements, permits access to environment variables, and more.

Advantages of declarative pipelines

- Syntax checking is performed at runtime in declarative pipelines. Explicit error messages are reported to the user before the execution is started.
- Users can link their declarative Jenkinsfile using a built-in API endpoint or a CLI command.
- Declarative pipelines offer extensive support for Docker pipeline integration. Users can choose to run all stages within a single container or each stage in a different container.
- The visual pipeline editor can be used to write declarative code in a user-friendly manner. The pipeline syntax snippet generator is a useful tool to auto-generate code snippets.
- Declarative syntax allows users to introduce conditional logic, including taking actions depending on the success or failure of a step or skipping a stage based on the value of a variable.
- The When directive can be used to skip over steps if certain conditions are not met.

Disadvantage of Declarative Pipeline

- Developers who have traditionally injected complicated business logic into pipeline code may struggle with certain limitations of declarative pipelines.

- For organizations that have been dealing with scripted pipelines for a long time, migrating from scripted to declarative code can be time-consuming and error-prone.
- It's impossible to inject blocks of declarative code inside a scripted pipeline. This limits cross-pipeline support.
- In a declarative pipeline, a reference to a Groovy script or a Java API will result in a compilation error.

Syntax

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                // Execute Some steps here
            }
        }
        stage('Test') {
            steps {
                // Execute Some steps here
            }
        }
        stage('Deploy') {
            steps {
                // Execute Some steps here
            }
        }
    }
}
```

Pipeline concepts

The following concepts are key aspects of Jenkins Pipeline, which tie in closely to Pipeline syntax.

1. `Pipeline`: A Pipeline is a user-defined model of a CD pipeline. A Pipeline's code defines your entire build process, which typically includes stages for building an application, testing it and then delivering it. Also, a `pipeline` block is a key part of Declarative Pipeline syntax.

2. `Node`: A node is a machine which is part of the Jenkins environment and is capable of executing a Pipeline. Also, a `node` block is a key part of Scripted Pipeline syntax.

3. `Agent`: is Declarative Pipeline-specific syntax that instructs Jenkins to allocate an executor (on a node) and workspace for the entire Pipeline.

4. `Stage`: A stage block defines a conceptually distinct subset of tasks performed through the entire Pipeline (e.g. "Build", "Test" and "Deploy" stages), which is used by many plugins to visualize or present Jenkins Pipeline status/progress.

5. `Step`: A single task. Fundamentally, a step tells Jenkins *what* to do at a particular point in time (or "step" in the process). For example, to execute the shell command `make`, use the `sh` step: `sh 'make'`. When a plugin extends the Pipeline DSL, that typically means the plugin has implemented a new *step*.

Declarative Pipeline Examples

We will be moving forward with Declarative pipeline as it is mostly in use now.

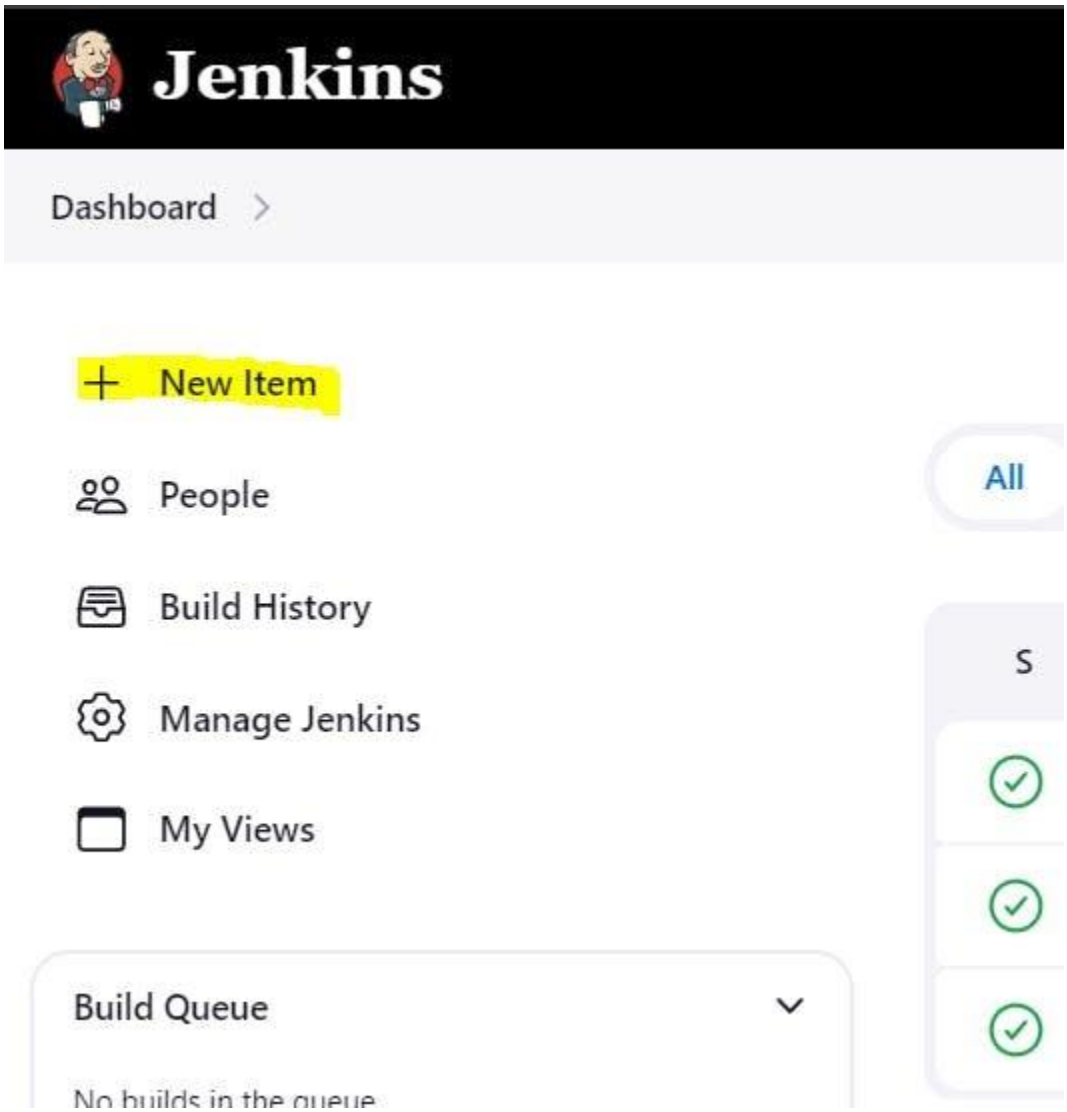
EXAMPLE — 1

In this , we are writing pipeline syntax direct in Pipeline.

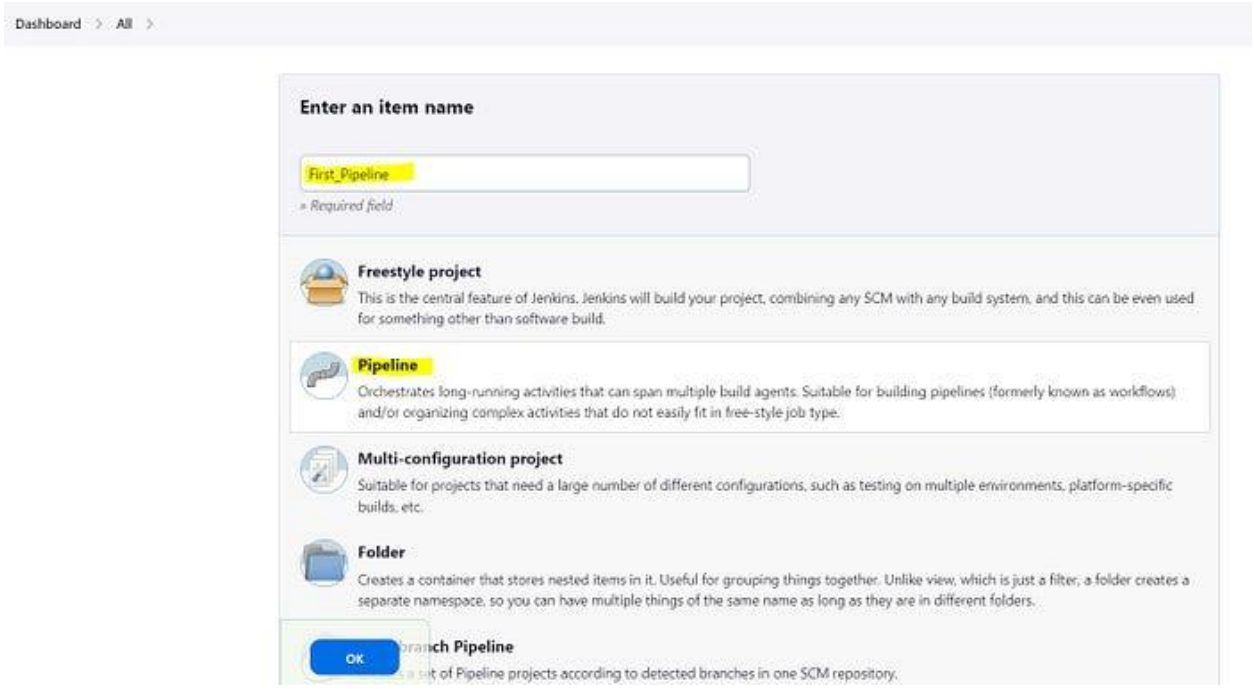
Step 1: Start your EC2 instance and connect your terminal.

Step 2: Login to your Jenkins via browser using `<Your_Public_IP:8080>`.

Step 3: Click on “New Item”.



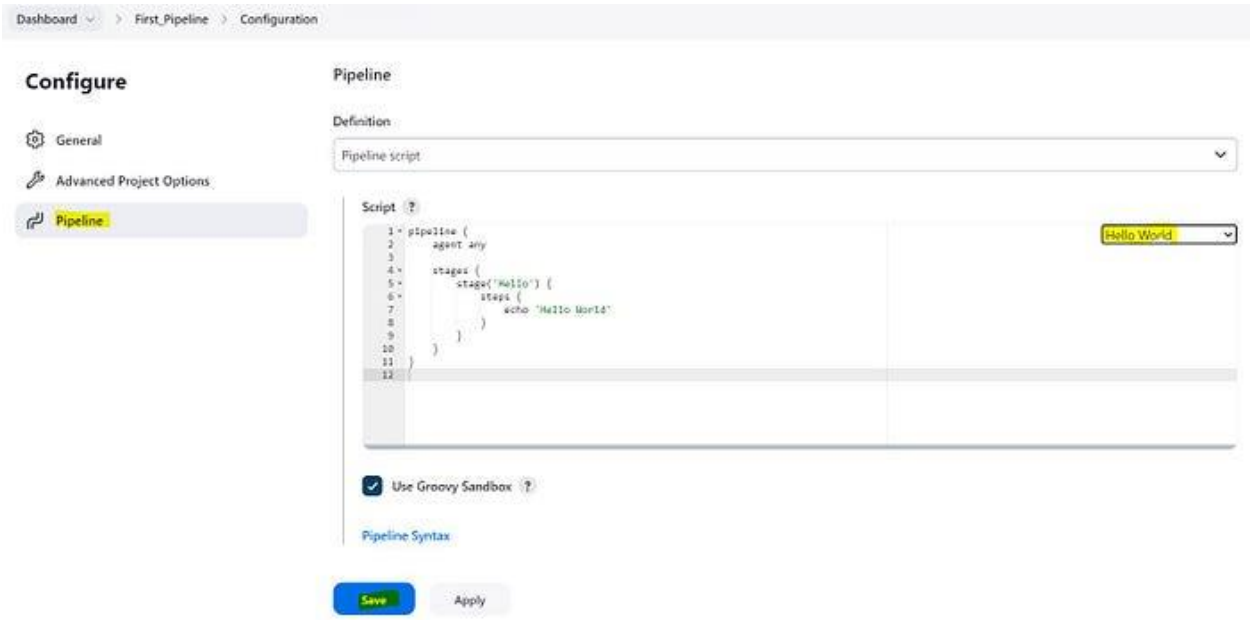
Step 4: Enter the name of the item as “First_Pipeline”. Click on Pipeline and click Ok.



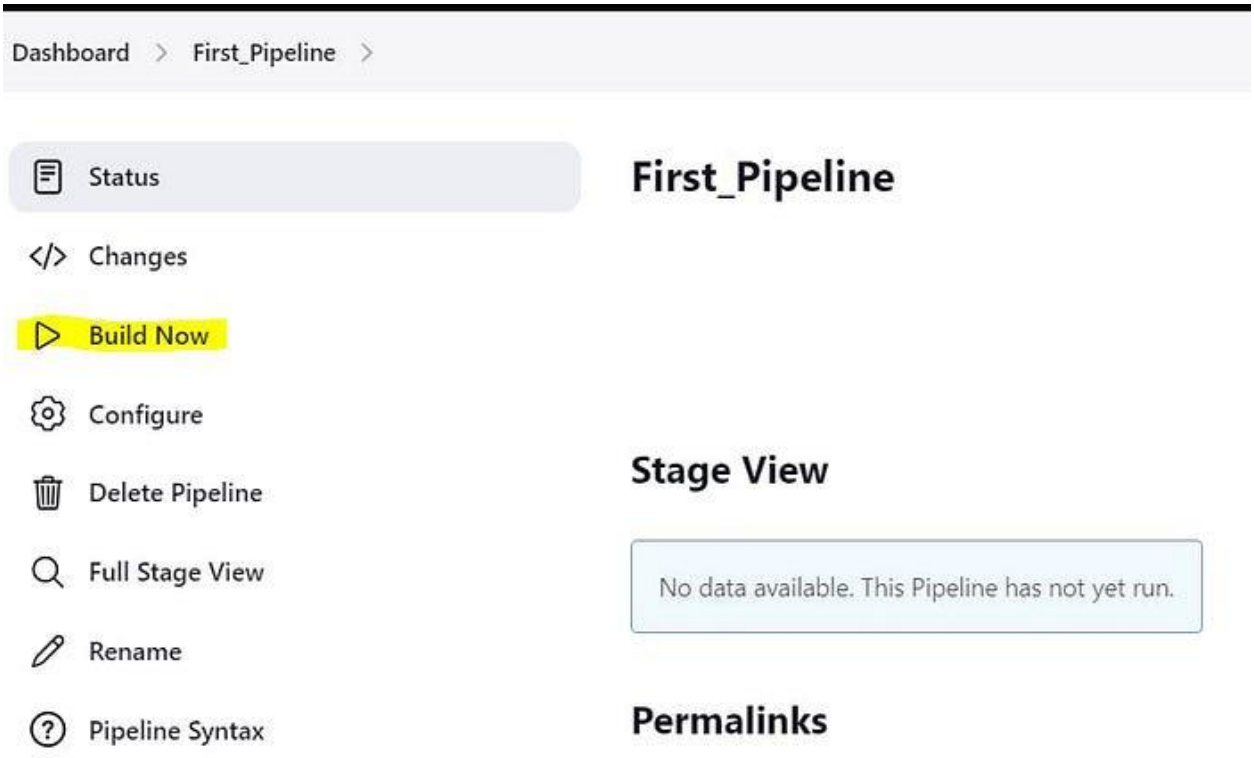
Step 5: Go to the end of the page, in the Pipeline section: Type the declarative pipeline script for Hello World as shown below and click on save.

Here the script is defined within the pipeline block.

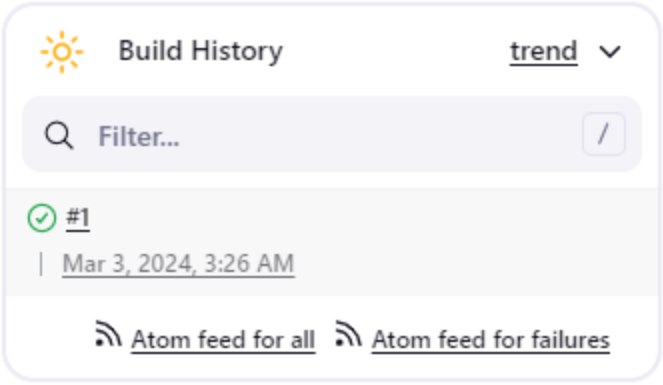
```
pipeline {
    agent any

    stages {
        stage('Hello') {
            steps {
                echo 'Hello World'
            }
        }
    }
}
```

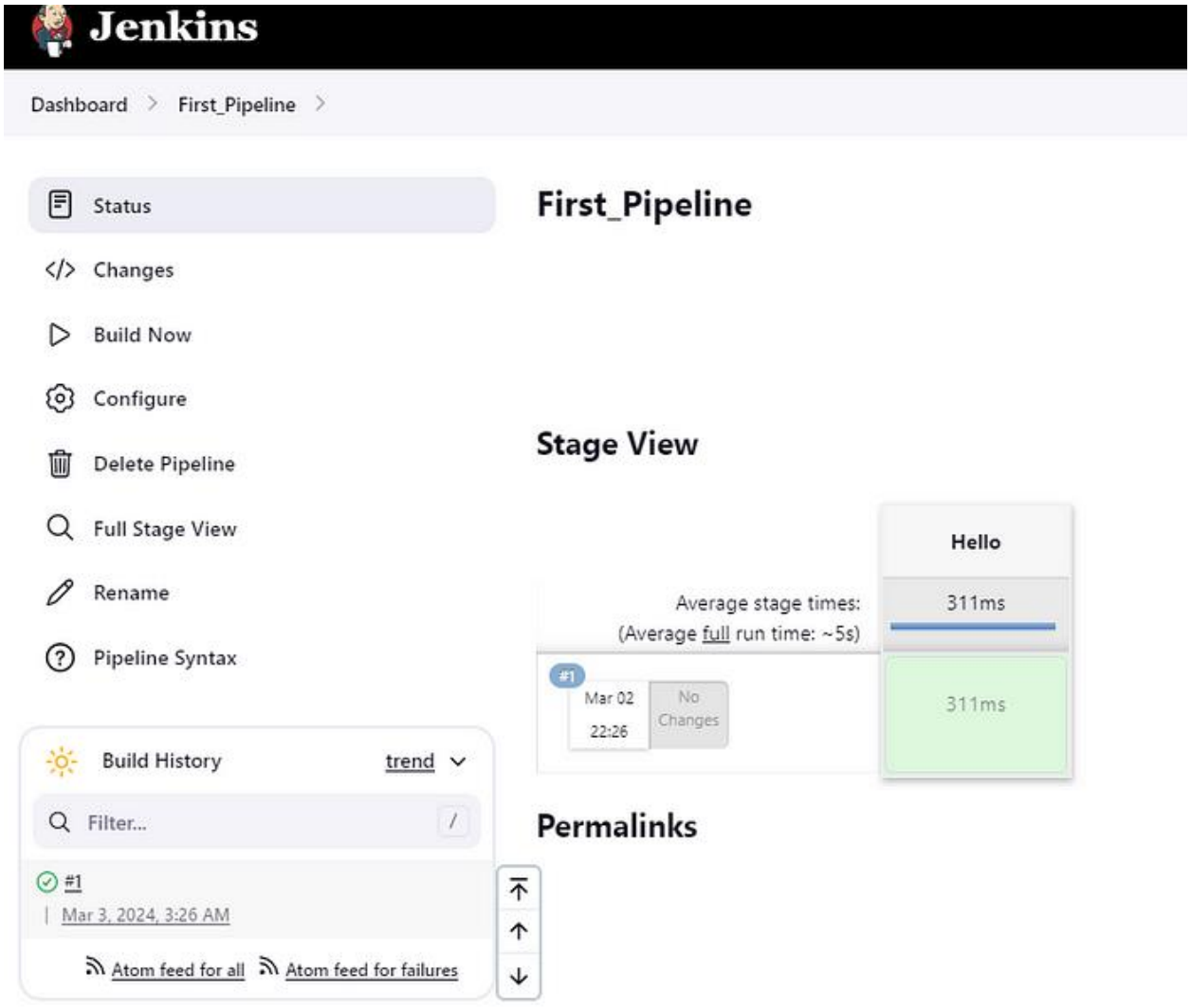


Step 6: Click on “Build Now”.

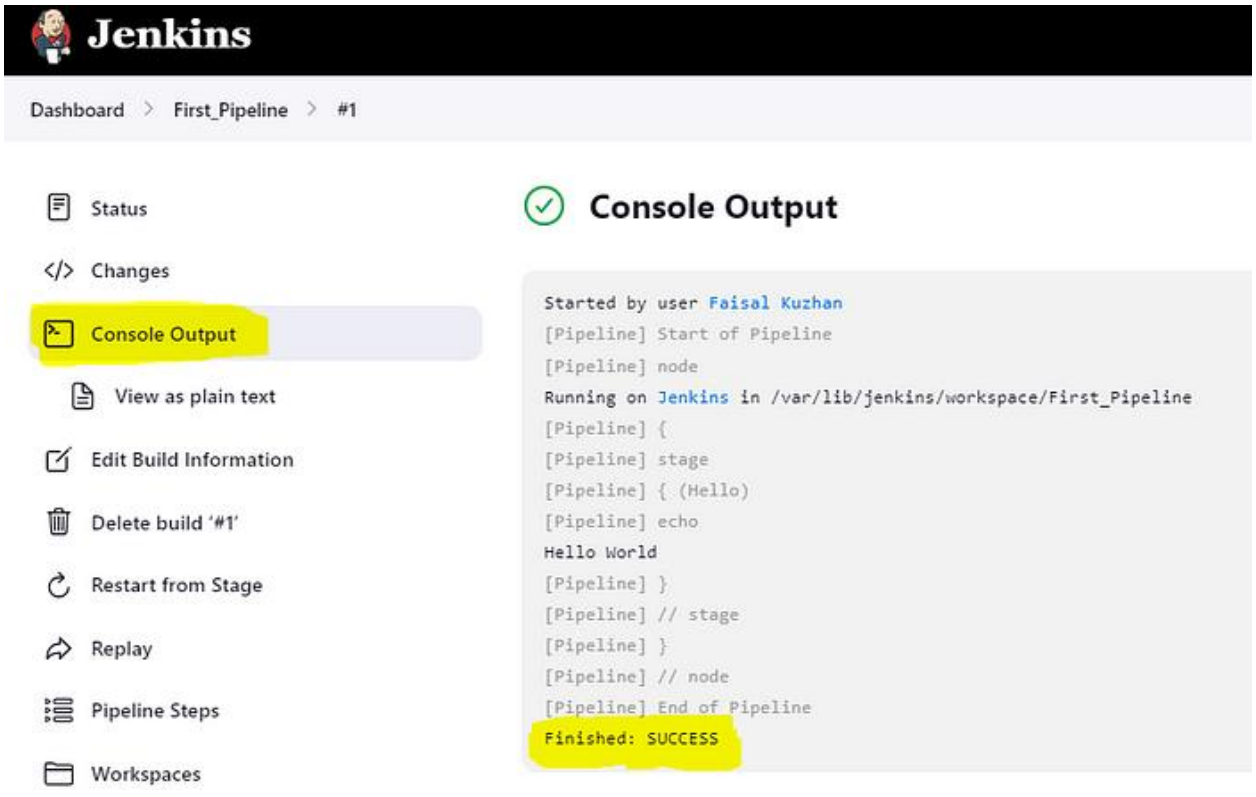


Step 7: The below image shows that the job is to build And it’s Successful.





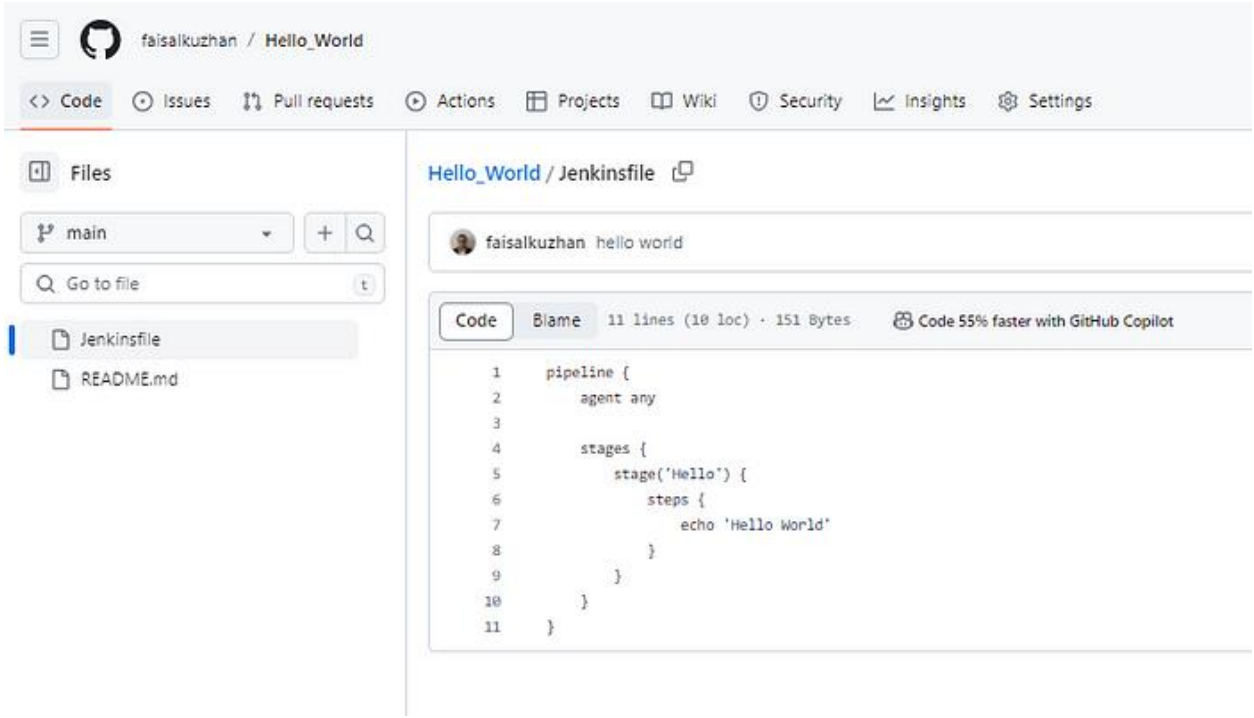
Step 8: Click on “Console Output” to see the complete Output.



EXAMPLE — 2

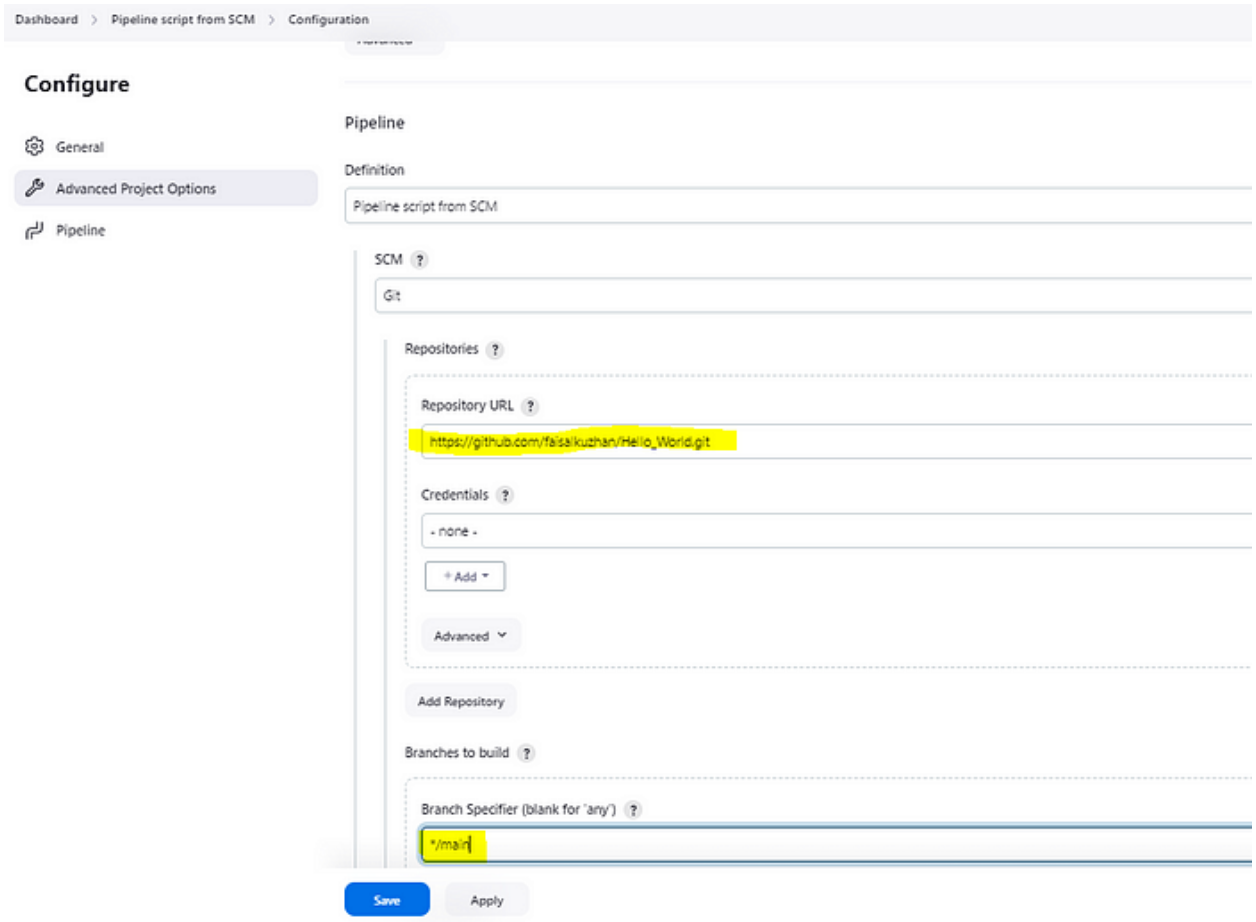
In this, we will write a Jenkinsfile and use that in our pipeline.

Step 1: Login to your GitHub and write a Jenkinsfile. We are using the same code used above.

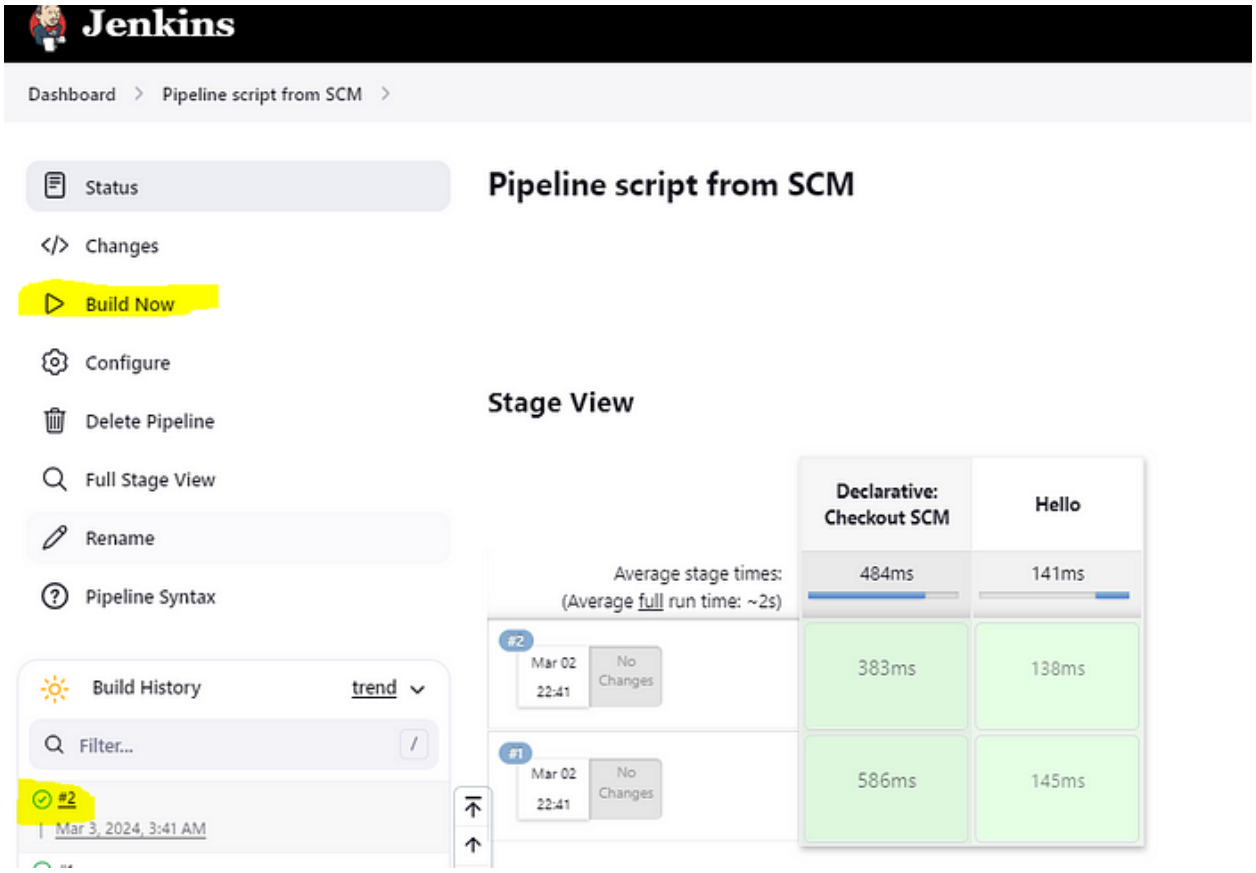


Step 2: Login to your Jenkins and Create a pipeline.

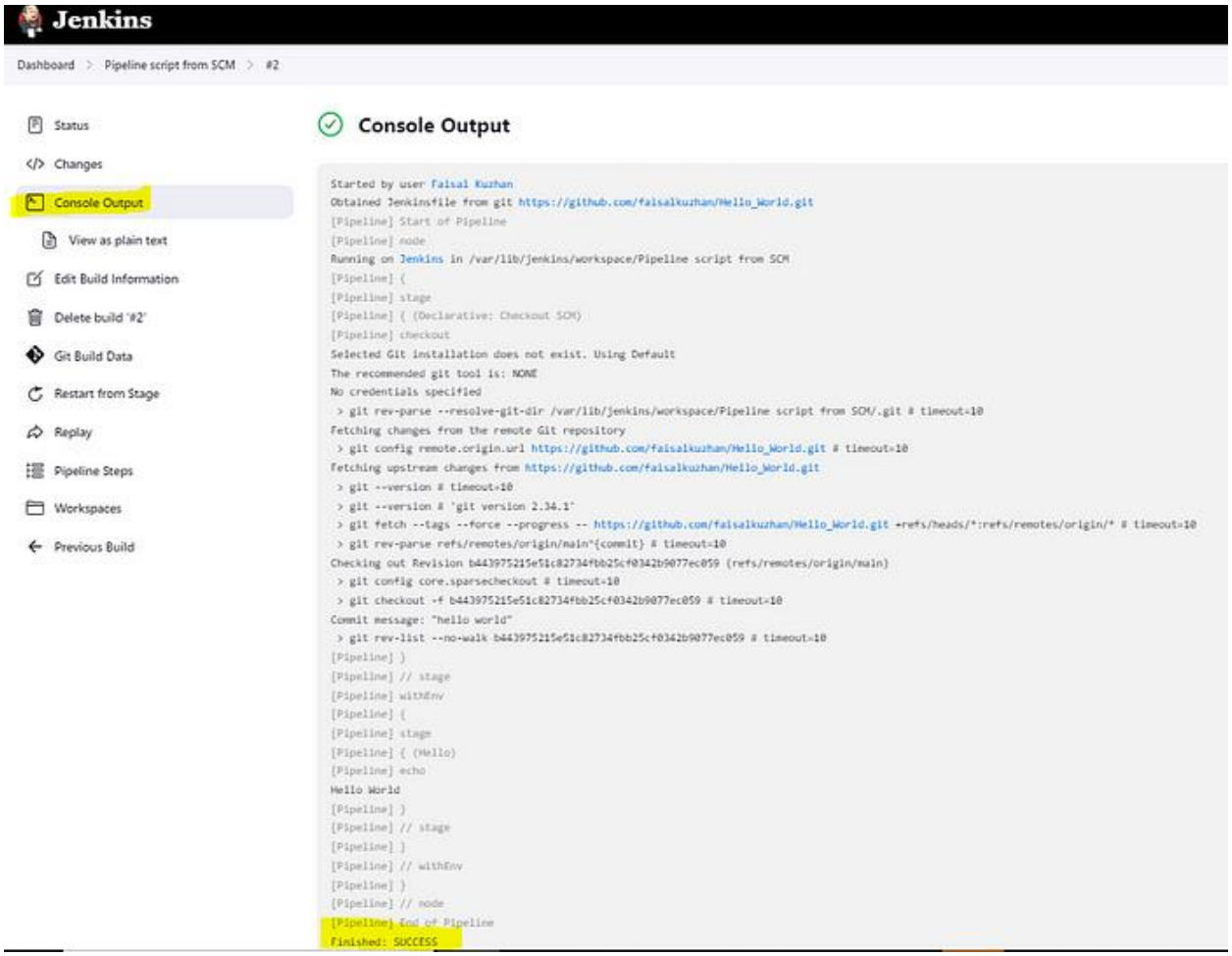
Step 3: Now In pipeline section, select “Pipeline script from SCM”. And in SCM, select “Git”.
Coy the “Repository URL” from GitHub and paste it in the “Repository URL” under Repositories Section and specify the “branch name” where the Jenkinsfile is located and Click on Save as shown below.



Step 4: Now click on “Build Now”. And you can see pipeline is build and it’s successful.



Step 5: Click on “Console Output” to see the complete Output.



So, these are the two examples, you can build your declarative pipeline.

Conclusion

In conclusion, a `pipeline` in the context of continuous integration and continuous delivery (CI/CD) is a set of automated processes that allow for the efficient and reliable delivery of software. Jenkins, a popular automation server, supports the definition of pipelines using Jenkinsfile, which can be written in either scripted or declarative syntax.

A `Jenkinsfile` serves as a configuration file that defines the pipeline stages, steps, and their relationships.

`Scripted pipelines` use a more flexible, imperative scripting language for defining pipeline logic, while `declarative pipelines` use a more structured and concise syntax for a more opinionated and streamlined approach.

`Scripted pipelines` offer greater flexibility and customization, allowing users to write arbitrary code in a Groovy-based scripting language. On the other hand, `declarative pipelines` provide a more straightforward syntax with predefined constructs for common use cases, promoting a more declarative and versionable approach to pipeline definition.

Ultimately, the choice between `scripted` and `declarative` pipelines depends on the specific needs and preferences of the development team. Both approaches aim to streamline and automate the software delivery process, contributing to more efficient and reliable software development practices.

What are the Jenkins Features?

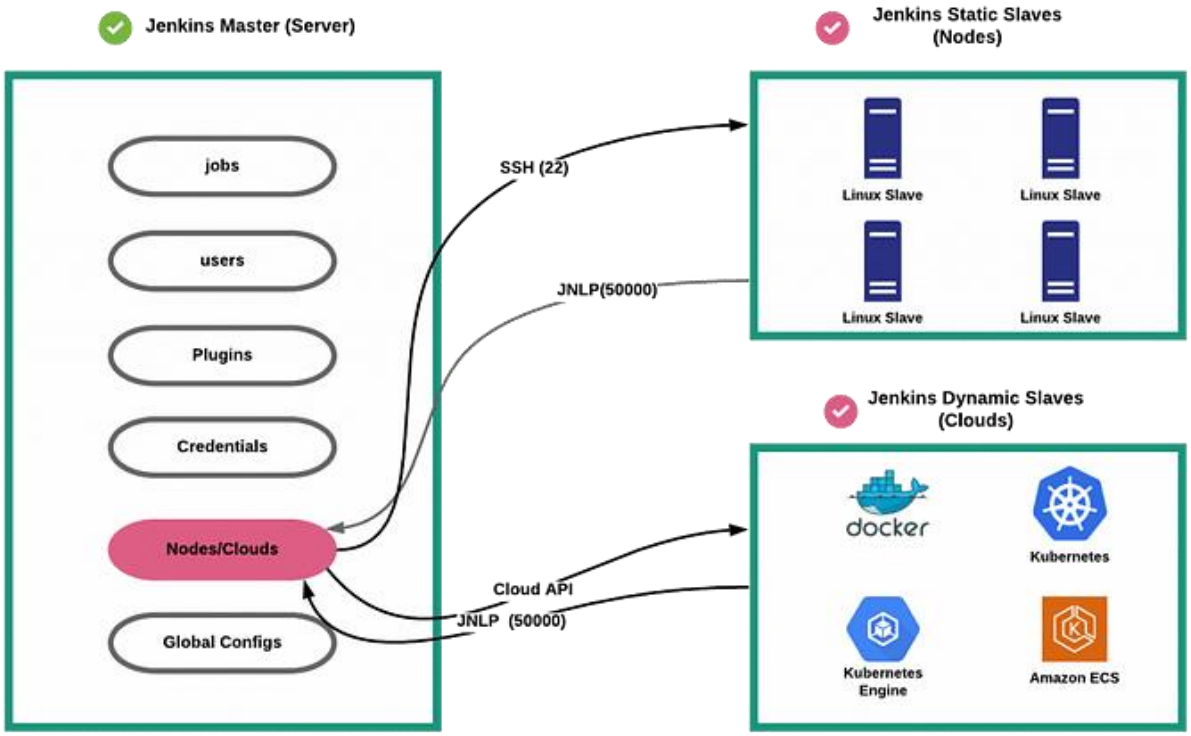
Jenkins offers many attractive features for developers:

- Easy Installation
- Jenkins is a platform-agnostic, self-contained Java-based program, ready to run with packages for Windows, Mac OS, and Unix-like operating systems.
- Easy Configuration
- Jenkins is easily set up and configured using its web interface, featuring error checks and a built-in help function
- Available Plugins
- There are hundreds of plugins available in the Update Center, integrating with every tool in the CI and CD toolchain.
- Extensible
- Jenkins can be extended by means of its plugin architecture, providing nearly endless possibilities for what it can do.
- Easy Distribution
- Jenkins can easily distribute work across multiple machines for faster builds, tests, and deployments across multiple platforms.
- Free Open Source
- Jenkins is an open-source resource backed by heavy community support.

As a part of our learning about what is Jenkins, let us next learn about the Jenkins architecture

Jenkins Architecture:

The following diagram shows the overall architecture of Jenkins.



Following are the key components in Jenkins

1. Jenkins Master Node
2. Jenkins Agent Nodes/Clouds
3. Jenkins Web Interface

Let's look at each component in detail.

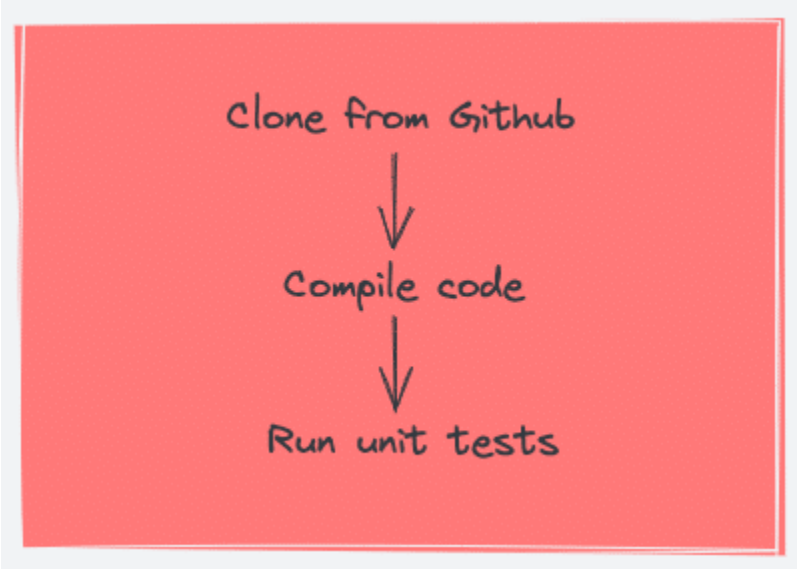
Jenkins Master (Server)

Jenkins's server or master node holds all key configurations. Jenkins master server is like a control server that orchestrates all the workflow defined in the pipelines. For example, scheduling a job, monitoring the jobs, etc.

Let's have a look at the key Jenkins master components.

Jenkins Jobs

A job is a collection of steps that you can use to build your source code, test your code, run a shell script, run an Ansible role in a remote host or execute a terraform play, etc. We normally call it a Jenkins pipeline.



Jenkins Plugins

Plugins are community-developed modules that you can install on your Jenkins server. It helps you with more functionalities that are not natively available in Jenkins.

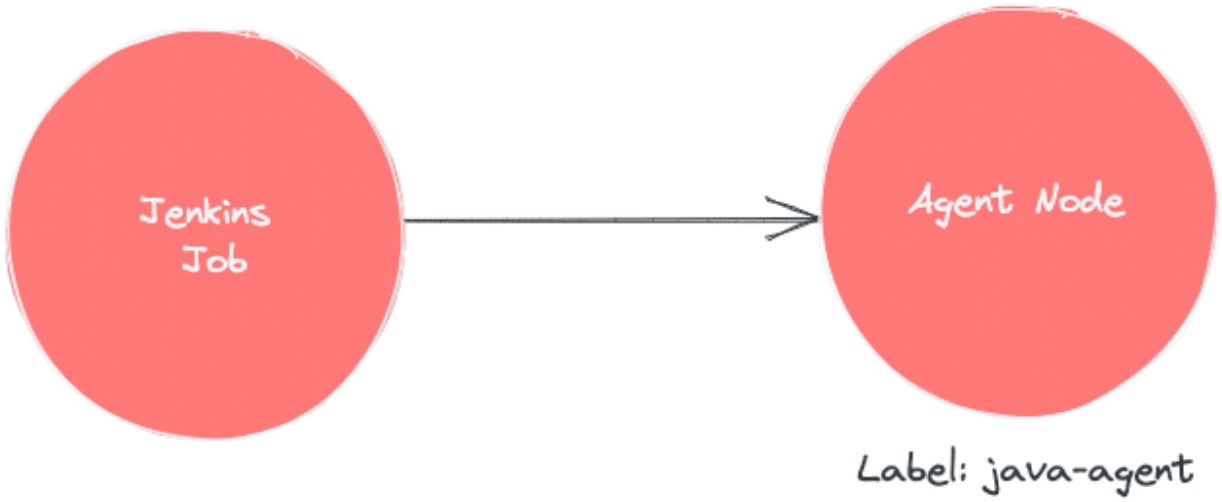
For example, if you want to upload a file to s3 bucket from Jenkins, you can install an AWS Jenkins plugin and use the abstracted plugin functionalities to upload the file rather than writing your own logic in AWS CLI. The plugin takes care of error and exception handling

Jenkins Nodes/Clouds

You can configure multiple agent nodes (Linux/Windows) or clouds (docker, kubernetes) for executing Jenkins jobs. We will learn more about it in the agent section.

Jenkins Agent

Jenkins agents are the worker nodes that actually execute all the steps mentioned in a Job. When you create a Jenkins job, you have to assign an agent to it. Every agent has a label as a unique identifier.



When you trigger a Jenkins job from the master, the actual execution happens on the agent node that is configured in the job.

Jenkins Nodes/Clouds

You can configure multiple agent nodes (Linux/Windows) or clouds (docker, kubernetes) for executing Jenkins jobs. We will learn more about it in the agent section.

Jenkins Master-agent Connectivity

You can connect a Jenkins master and agent in two ways

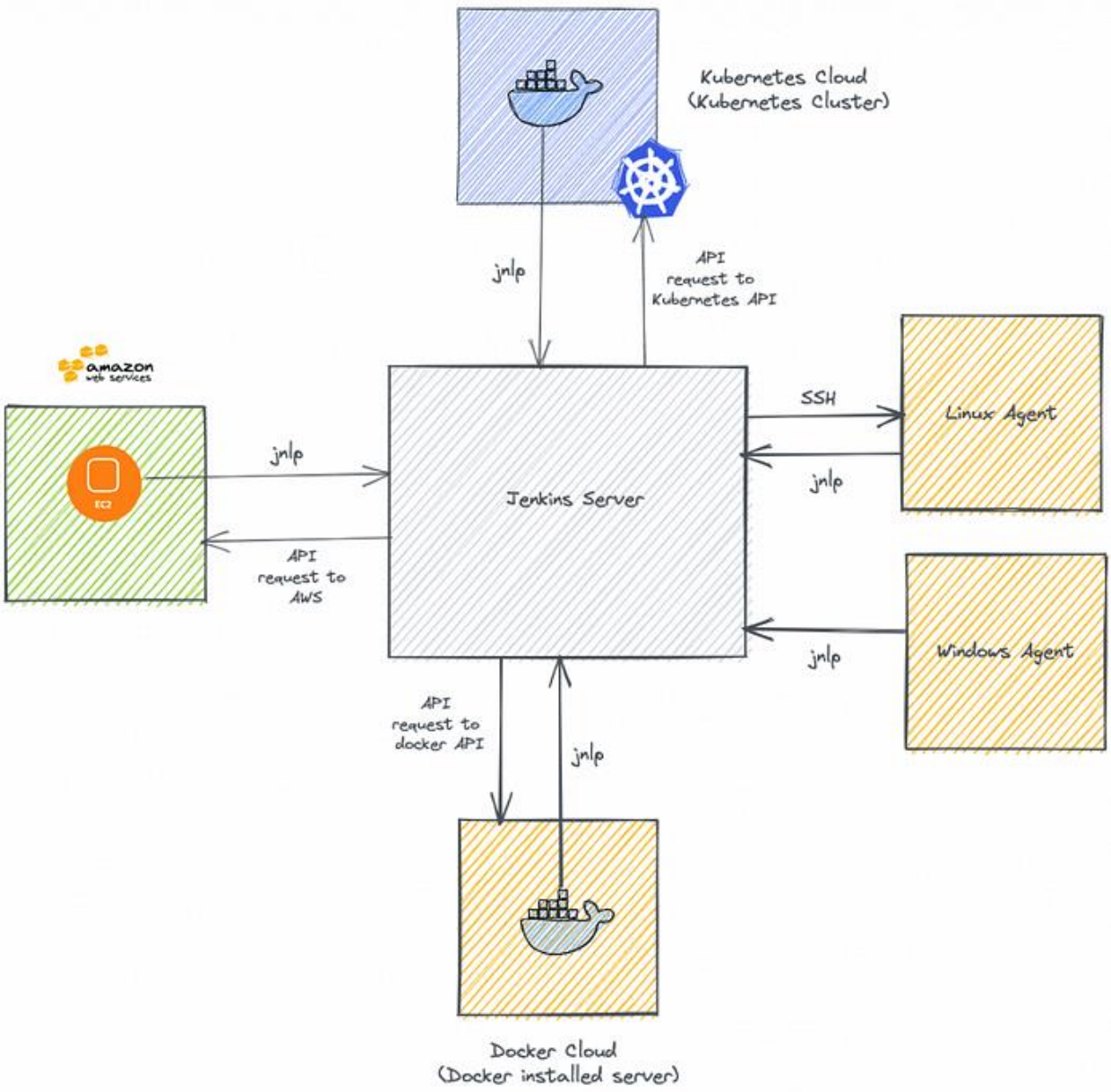
1. **Using the SSH method:** Uses the ssh protocol to connect to the agent. The connection gets initiated from the Jenkins master. Ther should be connectivity over port 22 between master and agent.
2. **Using the JNLP method:** Uses java JNLP protocol (Java Network Launch Protocol). In this method, a java agent gets

initiated from the agent with Jenkins master details. For this, the master nodes firewall should allow connectivity on specified JNLP port. Typically the port assigned will be 50000. This value is configurable.

There are two types of Jenkins agents

1. **Agent Nodes:** These are servers (Windows/Linux) that will be configured as static agents. These agents will be up and running all the time and stay connected to the Jenkins server. Organizations use custom scripts to shut down and restart the agents when is not used. Typically during nights & weekends.
2. **Agent Clouds:** Jenkins Cloud agent is a concept of having dynamic agents. Means, whenever you trigger a job, a agent gets deployed as a VM/container on demand and gets deleted once the job is completed. This method saves money in terms of infra cost when you have a huge Jenkins ecosystem and continuous builds.

The following image shows a high-level view of different types of agents and connectivity types.


Kubernetes Cloud
(Kubernetes Cluster)
jnlp
API
request to
Kubernetes API
amazon
web services
EC2
jnlp
API
request to
AWS
Jenkins Server
SSH
Linux Agent
jnlp
jnlp
Windows Agent
API
request to
docker API
jnlp
Docker Cloud
(Docker installed server)